

박지원 포트폴리오

Unity / C# Client Programmer

2026



이력서

- 이름 : 박지원
- 성별 : 남
- 생년월일 : 1996.03.16
- 🏠 : 서울특별시 동대문구 장안동
- 📧 : pjw960316@naver.com
- 📞 : 010-4761-3371
- 📄 Github : https://github.com/pjw960316/Unity_Client_Programmer

송실대학교 컴퓨터학부

2016.03 ~ 2022.02 : 컴퓨터학부 학사

2017.04 ~ 2019.01 : 군 복무



풀어비스 인턴

2021.12 ~ 2022.02 : 플랫폼 프로그래밍 2팀



넥슨게임즈 넥토리얼 2기 및 정규직

2022.11 ~ 2024.04 : 제우스 프로젝트_Unity 클라이언트 프로그래머

- ✈ Project ZEUS => GODSOME
- ✈ GODSOME Official Trailer

목차

과거의 회사 경험을 통해 문제 의식을 발현

- 이전 직장생활에서 알게 된 저의 문제점 네 가지로 구성되어 있습니다.

첫 번째 해결과정 : 게임 개발 설계 기준 세우기

- 설계 기준을 디자인 패턴을 공부
- 개발한 게임의 간단한 설명이 있습니다.
- 왜 이런 게임을 개발하게 되었는지
- 첫 번째 해결과정에서 설계한 구조를 실제로 어떻게 구현했는지.
- 정답은 아님 아직도 부족함. 근데 이제 어떻게 해결할지가 보이고.

두 번째 해결과정 : c# 구현력과 AI

세 번째 해결과정 : 메모 -> 기록

마지막 해결과정 : 다시 일하기 위한 나만의 지도

후기

과거의 회사 경험을 통해 문제 의식을 발현

프로젝트 방향성

- 저는 유니티 클라이언트팀 신입 막내로 커리어를 시작했습니다. 그 전에 두 달 동안의 인턴 기간은 사실 큰 배움을 얻진 못했습니다.
 - 팀은 저 포함 주니어 개발자 3명, 그 외에는 모두 8년차 이상 시니어로 총 15명이었습니다.
- 1년 4개월의 짧지만 길었던 재직 기간동안 마주했던 저의 문제점을 우선 적고 지금은 해결한 방향으로 포트폴리오를 구성했습니다.

퇴사 직전 면담 때, 혼자 슈팅게임을 만들 수 있느냐?에서 저는 '아니요'라고 답했습니다.

- 당장 코드 밑바닥 설계부터 자신이 없었다. 당시의 입사 포폴은 그냥 모든 걸 monobehaviour 아래에 박았었으니 만들었던 거고.
- 매 번 base class의 코드가 몇 천줄인 걸 하나씩 천천히 읽어 볼 생각, perforce의 commit 로그, 히스토리 그게 참 관리 되어 있었는데 못 봤었다.
- 팀에서 내게 요구했던 건 그건데. 나는 조금해서 당장 메서드 하나라도 막 생산만 하는...
- base를 이해를 못하니, 중복을 만들고, 스파게티를 만들고, 총체적 난국. 근데 당시에 어디서 부터 잘 못 됐을까? 근데 사실 인턴 때 부터 알고 있었다. 회피했던 것 뿐이다. 내가 성장하지 않았던 거를. 그 고민의 결과가 팀장님의 저 질문에서 아니라고 말했던 거 같다.

기초가 약했고, 아니 실무 레벨에서는 거의 없는 수준이었습니다.
그러나, 해야 할 게 너무 많으니 오히려 회피했던 것 같습니다.

- 4년 전공자여도 항상 불안했다. -> 나는 스스로에게 물었을 때 코딩을 잘하나? 아니다.
- github 링크 -> 대학 시절
- 인턴을 마치고 정직원이 됐을 때 높은 난이도의 업무를 받고는 싶었으나 또 해결하는 게 두려웠다.
 - deep한 코드를 읽으려 했는가?
 - 욕심은 또 있다. 그러나 너무 조금했다.
- 역으로 생각하는 힘이 있었을까?
 - 내가 이 부분이 부족한데 그걸 채우려는 노력을 했었나?
- 천천히, 꾸준히와 가장 대척점에 있었다.
 - 할 건 많고 리스트업은 하지만 안했다.

과거의 회사 경험을 통해 문제 의식을 발현

메모만 하고 기록으로 가공하여 제 것으로 만든 적이 없었던 것 같습니다.

- 인턴 중간 평가 때 팀장님께 혼난 기억이 있다
 - 너는 뭘 공부했니? 나도 뭔가를 많이했었다.
 - 저는 vscode에 열심히 기록하고 있어요 (사실 메모 수준)
 - 보여줄래?
 - 어... 그니까... 적기만 하고 본 적이 없으니... 모바일로 아이디 로그인도 못했던. 실제로 뭔가 한 것도 별로 없었고 제대로 정리한 거도 없고.
- 생각해보면 그냥 공부를 못했다. 매 번 90점은 잘 받아도 왜 100점을 못 맞을까?

감정에 나약했고, 꿈도 너무 빨리 이뤘다

- 대학 졸업전에도 인턴을 했고, 대학 졸업하고도 인턴 및 정직원을 했다. 이게 인생의 목표였었고 금방 이뤘다. 다음이 없었고 회사에서 잘해야지만 머릿속에 담았던 거 같다. 4개월의 크런치도 크게 힘들지 않았다고 착각했다. 그래서 버텼다.
- 회사가 아닌 삶 전체의 지도가 없었다.
- 유복하게 자라왔고 살면서 큰 위기도 없었다. 그러다보니 무언가 혼자 해결해 본 적도 발버둥 치본 적도 없었다. 하지만 퇴사하고 그렇게 무던하게 살아온 거에 감사하면서도 그래서 나약했구나 싶었다.
- 그렇게 퇴사를 하고 가장 인상깊은 이야기 -> 그 정신력이면 또 같은 이유로 퇴사를 한다고 하더라.
- 도움을 받을 줄도 몰랐다. 돌이켜보면 팀장님과 파트장님 그리고 멘토님은 정말 어떻게 해서든 나를 성장시키려고 노력하셨다.
 - 심지어 팀장님은 마지막에 개인 발표시간도 1대1로 해주셨다. 그 때 했던 게 jeffery ritcher였다. 얼마 안했지만. 그래서 이 책은 나에게 큰 의미를 준다.
- 회사를 다닐 때 파트장님께 인정받고 싶었다. 그래서 혼나면 슬퍼하고 칭찬받으면 기뻐했다. 팀장님 면담을 하고 희망차면 또 기뻐하고, 절망적이면 며칠을 잠을 못잔다.
- 지금은 좀 단단해 진 거 같은 이야기를 (이건 좀 마지막에 들어감) -> 마치며에서 채움.



첫 번째 해결과정 : 게임 개발 설계 기준 세우기

1 문제 재정의 + 어떻게 해결해야 할지

- 보여지는 단순 기능은 많이 만들어도 결국 성장하지 않음을 깨달았습니다.
- 게임의 근간을 받치는 기본 코드, 가장 밑 바닥에 가장 호출이 많이 되는 베이스 클래스를 제대로 구현해보고 싶었습니다.
- 이 코드를 어떤 계층에 적어야 할 지, 이 클래스는 어떠한 책임을 가져야 할 지, 이 클래스는 누구와 대화를 해야 할 지에 대해 고민을 해보았습니다.
- 어떤 게 외부에 노출되면 위험한지,
- 유지보수가 잘 되는 코드가 어떤 코드일지, 새로운 기능이 추가될 때, 개발 도중 기획이 추가될 때 나는 쉽게 추가할 수 있는지? 오염이 적은 지에 대해 고민해보았습니다.

2 설계 없이 기능부터 만들던 방식 → MVP + Manager & Controller로 책임 분리

설계도

설명

- Model
 - 게임 오브젝트의 데이터를 관리하는 스크립트
 - 클라 & 서버 구조에서는 서버의 영역이라고 생각한다. 서버의 검증된 DB를 패킷으로 받기 때문이다.
 - 물론 클라도 일부를 캐싱해놓기 때문에 서버로부터 받는 데이터를 저장하는 클라 전용 Model
 - 데이터의 변경은 반드시 presenter를 통해 전달되어야 한다.
- presenter
 - view와 model은 서로 모릅니다.
 - 하지만 view는 데이터가 필요하고 model도 데이터의 변경 사항을 화면에 보여줘야 합니다. 그걸 presenter를 통해 대화를 합니다.
 - 또한 MVP가 결합된 하나의 게임 오브젝트가 됩니다. 게임 오브젝트는 크게 보면 필드 오브젝트와 UI입니다. 특히 필드 오브젝트 한 개만 존재하지 않습니다. 그래서 필드 오브젝트 관리하기 위한 싱글턴 매니저는 필연적으로 존재하고, 이 싱글턴 매니저는 Presenter와만 대화를 합니다.
- View
 - view는 아무것도 모르는 멍청한 영역입니다.
 - 오직 Presenter에게만 의존하며 presenter의 명령을 받으면 사용자에게 보여지는 View를 갱신하기만 합니다.
- Manager
 - 필드 오브젝트가 여러 개 존재하면 그 들을 관리해야 하죠. 예를 들면 클릭하면 타겟을 바꾸고, 이전 타겟을 롤백하는 기능 같이요. 그런거는 싱글턴으로 게임에서 한 녀석이 담당합니다.
 - 게임에는 다수의 싱글턴 Manager가 존재합니다. 그리고 그들끼리는 대화가 가능하죠. 그리고 Manager는 각각의 MVP 객체 (ex : 필드오브젝트, UI)와 대화를 하고 이 때 Presenter와만 대화를 합니다.
- Controller

- 매니저가 MonoBehaviour를 상속 받으면 의존성이 커집니다. 그러므로 Unity에서 동작하는 기능은 Controller에서, 그 외에 C# 영역은 Manager에서 하도록 둘 을 분리했습니다.

코드

```
int a = 10;
int b = 20;
Console.WriteLine(a + b);
```

3 하나를 고치려면 스크립트 여러개를 읽었던 기존의 방식 → 의존성 및 책임

양방향 의존성에서 단방향 의존성으로, 언젠가는 의존성을 끊기도 하는

- Manager와 Controller를 분리하면서 Manager도 Controller를 필드로 들고 있고, Controller도 Manager를 들고 있으니 구현은 쉬웠습니다. 하지만 서로 강하게 의존하고 있었죠.
- 둘은 각자의 책임이 존재합니다. Manager는 자신을 필요로 하는 MVP로 구성된 객체들에게 필요한 정보를 전달 해주어야 하고, Controller는 유니티 세상에서 Manager가 필요로 하는 데이터를 가공해서 전달해줘야 합니다. 그러면 사실 Controller는 가공만 하면 됩니다. 그리고 Manager에게 열어두고요. Controller가 public으로 메서드를 열어두면 외부에서도 접근 할 수 있지만, Controller는 MonoBehaviour 상속이므로 new가 되지 않고 이는 쉽지 않죠. 그러니 Manager가 해당 메서드가 사용 가능하고, 변경에 대해 안전합니다. 만약 어디서든 Controller도 접근이 되어 의존이 되면 변경이 너무 쉬워지기 때문에 관리가 되지 않습니다.
- Controller는 외부 여러 객체에 직접 노출되지 않고, Manager를 통해 제어되도록 제한했다. 다른 외부와 단절되어 있고요. 그렇게 안정성이 높아짐을 확인했습니다.
- 언제나 Manager를 통해서만 변경이 되므로 변경의 주체가 Manager 하나로만 종속되니 유지보수가 좋아집니다

클래스의 책임을 명확히하고 메서드는 하나의 기능만 다루는 게 좋습니다.

- 클래스의 책임을 분리하다보면 클래스가 자동으로 쪼개집니다.
- 그러다 보면 책임이 명확해지기도 합니다.
 - 이 클래스는 창세의 View를 담당하니, 간단한 이동, 간단한 애니메이션은 View지만 필드를 여기 두어도 되겠다.
 - 하지만 창세의 동작을 구현하기 위해 유저의 입력 처리는 창세 View 스크립트의 책임이 아니다. 이건 InputManager의 책임이다.

4회사 다니면서 디자인패턴 책만 3개 샀고 읽기만 -> 디자인 패턴에 대해서 알게 된

과거에 바라 본 디자인패턴

- 학교나 회사에서 디자인패턴 책은 항상 구매했었다. 매 번 싱글턴, 옵저버 같이 익숙한 패턴만 눈으로 읽고 넘어갔었다.

- 새로운 디자인패턴도 예제만 읽어보면 절실함이 없으니 그냥 뭐 이론이구나 하고 속 보고 넘어갔다.
- 파트장님께서 디자인패턴은 이론적으로 공부하는 게 아니라, 구현하다보면 어느덧 사용하고 있는 자신을 바라보는 거라고 했다. 당시에는 전혀 이해가 되지 않았다. 지금은 조금 이해가 됩니다.

롤토체스 게임 연구 과정과 연결 지은 생각

- 전략게임을 상위 랭크를 달성한 경험이 있습니다. 그 때 과연 왜 갑자기 잘해졌을까? 싶으면, 결국에 무수히 많아 보이지만 게임의 상황은 10가지 안에 존재했던 것 같습니다. 그래서 그 패턴들마다의 상황을 요약하고 그 상황마다 최적의 해를 도출하는 걸 즐겼습니다. 그리고 나 보다 좋은 답을 찾으려고 고수들이 만들어 놓은 정답에 가까운 플레이를 해석했죠.
- 그리고 이게 결국 디자인 패턴과 유사함을 느꼈습니다. 게임 개발에서도 접목을 해봤습니다.
- [🔗 게임 연구 문서](#)

디자인 패턴

- 생각해보면 개발 상황도 크게 보면 문제 상황이 비슷한 경우가 많습니다. 추상적으로요. 근데 매 번 똑같은 상황에 똑같이 실수 했었던 것 같습니다. 저만의 대응 패턴이 없었지요. 게임을 하면서 그걸 갖고 닦으니 실력이 빨리 늘었습니다. 공부와 개발과는 다르게 바로바로 결과가 보이는 게 장점이었고요.
- 책임을 분리하고, 아 매번 누가 대리로 생성을 해주면 생성과 해제의 관리가 쉽지 않을까 고민했음. 팩토리 패턴 모른 상태에서. 근데 그렇게 고민하고 개발을 해보니까 내가 한 게 팩토리 패턴인 걸 알게 됐음. 내가 view 생성시 awake의 콜백으로 PresenterMnager가 presenter 생성하는거!
- 추상적으로 같은 기능인데 데이터가 달라서 if-else를 무수히 많이 생성하게 되는 경우가 있습니다. 특히 input의 경우 다양한 게 들어오지만 결국 인풋을 처리한다는 기능으로 동일하고요. 그러다가 어떻게 하면 1,000개가 들어와도 처리가 될까? 결론적으로는 분기는 당연히 한 번은 필요합니다. 대신 handler를 이용하면 또한 맵핑을 하면 처리가 되지요. 그리고 그거에 가장 정답에 가까운 게 전략패턴이고요.
- 부족하지만 내부 구조를 구현하려고 삽질을 좀 많이 했습니다. 그러다 보니 구현을 많이 못하고 설계만 하기도 했죠.
- 하지만 그 과정에서 클래스 구조의 확장성 문제를 직접 마주했고, 의존성 방향, 자료구조, 상속 구조, 다형성 적용 방식을 바꿔가며 여러 설계 시도를 했습니다.
- 예전에는 개발과정 없이 그냥 책만 읽었으니 도움이 안 됐습니다. 그러나 제가 삽질하며 도달한 구조를 책에서 찾아보니 특정 디자인 패턴과 닮아 있다는 것을 알게 되었고, 제가 삽질한 과정과 비교를 해봤습니다.
- 그러다 알게 된 건 디자인 패턴도 정답이 아니고 대단한 기법도 아니다. 결국 모두 trade-off고 장단이 있다는 것일요.
- 분기를 줄이고 싶어서 전략패턴을 만들었지만 결국 코드가 복잡해지고, 움저버 패턴을 만드니 의존성을 줄지만 디버깅이 어렵고. 하지만 분명 계속 개발하면서 이걸 신경쓰면 저만의 패턴이 생기는 거죠.

코드 예시

쉬운 기능을 막 쓰는 건 위험함 -> UniRx 남발 및 콜백 지옥 창시

- 신입은 기능 구현이 우선이다. 그리고 시니어라고 다를까?
- 하지만 나는 모든 기능을 Rx에 담아버렸다. UniRx를 제대로 이해하고 쓰지 않았다. 그냥 subscribe하고 OnNext()하면 다 돌아가니까
- 인턴 마치고 첫 업무가 팀 전체의 UniRx, Action, MonoObserver(custom), Observable을 정리하는 것
- 팀장님의 의도는 Event System 코드를 보고 이해해라.
- 지금은 과한 Rx도 무겁다고 생각한다. event Action에서 예전에 왜 event를 붙이지? 근데 지금은 안다. 외부에서 호출이 자유로우면 의존성이 쉽게 열려버린다. 근데 event를 쓰면 구독 +-만 할 수 있다. 호출은 못한다. 저것도 가끔은 불안하다. 하지만 최소한 이벤트의 실행 주체까지 위임하지는 않는거다.
- 혼자 개발하다 보니 이런 게 보이고, 내가 UI단에서 남발한 UniRx의 public들이 얼마나 위험했을까. 그리고 형님들은 절대 재사용하지 않았을 거다.
- 유지보수가 가능한 게임을 만들려고 노력했다.
- 실무에서는 마감이 있으니

3 달라진 점 -> 이거 일부는 2번 해결과정에 적자.

- 과한 설계의 독이 생겼다.
 - 좋은 설계도 있고 정답에 가까운 설계도 있지만 결국 정답은 없는 것 같다.
 -
- F12와 콜스택을 보는 능력이 좋지 않았다. -> 어차피 베이스 코드는 잘 모르니까 매번 콜스택의 상단부분만 본 것
- 예전에 시니어 분이 인풋 시스템을 만드셨고 FSM을 쓰셨습니다. 상태 기반으로 기능이 동작하는. 팀에서는 반발도 있었죠. 굳이 어렵게 하고, 굳이 무거운 기능을 만드냐. 근데 기억에 우리 팀은 새로운 인풋 기기에 대한 욕망이라 (2년 이상)가 많았던 걸로 기억합니다. 예전에는 시니어 분이 화이팅! 이라고 넘어갔고 나와는 먼 이야기니까 선을 그었습니다. 그러나 지금은 아 왜 FSM을 썼을까? 인풋이 엄청 대응이 되네 그리고 유니티도 API를 지원해주니 편하구나. 이런 게 보입니다. 이전에는 저 업무가 내게 올일은 없다. 그러나, 언젠가 오면 어쩌지? 나 못하는데 였으면. 이제는 적어도 왜 필요한지를 알게 되니 관심을 갖고 도전해 볼 영역이라 생각합니다.
- LINQ도 래핑해서 쓰고,
- 문제는 코드를 추가할 때마다 제가 어떤 기준으로 책임을 나누고,
- 어떤 방향으로 의존성을 흘려야 하는지 명확한 판단 기준이 없었다는 점이었습니다.

기능 하나를 구현할 수는 있었지만, 그 기능이 시간이 지나도 유지보수 가능한 구조인지, 다른 시스템과 결합이 과해지지는 않는지, 나중에 확장할 때 어디를 수정해야 하는지까지 판단하지 못했습니다.

그래서 퇴사 이후 처음 다시 잡은 기준은 '무엇을 만들 것인가'보다 '어떤 기준으로 코드를 나눌 것인가'였습니다.

- 기술적 영역
- MVP, Dependency, Manager-Controller, Presenter, Model/View 분리, 의존성 방향 같은 내용이 들어가면 좋습니다
- mvp + manager + controller 구조도

- 구현 -> 버그 3개 이상 -> 버그 고치기 -> 잘못된 깨달음 -> 돌아가니 방치 -> JIRA 생성 -> 야근 업무
- 구현 -> 버그 1개 -> 버그 고치기 (난이도가 조금 쉬워짐) -> 리팩토링 할 건 여전히 보이나 방치될 때도 있지만 ->
- 과설계
- 욕심은 많아서 상위 타입으로 올려버리고 리팩토링 (완벽주의가 심했어서 구현은 안하고 매 번 이런 거 만 했다.)
-> 어설픈 최적화는 독이되고 -> 실제로 버그를 많이 만들었다.
- 공부하는 방식을 전혀 몰랐던 거 같다 이제 까지는.

후기

- 처음 만들게 된 계기
- 실제로 쓰이는 게임 만들기
- 기능 캡처

GitHub 링크

-  [GitHub_Unity_MVP Pattern](#)
-  [GitHub_디자인 패턴](#)
-  [GitHub_Toy Project](#)
-  [GitHub_Toy Project Script Folder](#)



세 번째 해결과정 : C# 구현력과 코딩테스트 그리고 AI 활용

1 현업에서 요구하는 수준의 기초가 부족했음을 깨달았습니다. 그리고 AI 시대에 이는 더 큰 문제점으로 다가 올 수 있겠다고 생각합니다.

- 인턴 기간에는 GPT가 지원되지 않았고 운이 좋게도 정직원 기간에는 GPT가 지원 됐었습니다.
- AI가 주는 장점은 확실합니다. 멘토님께 물어보기 모호한 질문도 물어볼 수 있고, 구글에 질문해서 양질의 답변을 고르는 시간도 줄일 수 있습니다.
- 당시에도 AI의 코드를 복붙은 하지 않았습니다. 그러나 팀원분들과 AI를 통해 지식을 전파받고 좋은 이야기를 들 어도 기초가 늘지 못했죠.
- 그리고 이 때 내가 코드몽키가 되어가고 있구나. 이 사실이 들키지 않으면 좋겠다. 그래도 고치지 못했으나 자각 은 했다.
- 과거의 복붙 개발과 현재의 AI 딸깍 개발은 겉으로는 달라 보인다. 하지만 '이해하지 못한 코드를 가져와서 동작 만 시키고 넘어간다'는 점에서는 같은 상태일 수 있다.
- 과거의 경험이 있기에 오히려 GPT로 인해 편하게 먹는 지식이 더 먹기 어려운 것 같습니다.
- 결국 회사 업무는 기획 문서 -> 개발 회의 -> 개발 계획 세우기 -> 구현 & 중간 리비전 서밋 -> 최종 구현 -> 검 토 -> 버그 수정 -> 배포입니다.
- 구현과 버그 수정에 대해서 이제는 쉽게 조언을 받을 수 있습니다. 그러나,,,

2 오랜 기간 AI를 사용했지만 그래도 여전히 제가 알고 물어보는 게 중요한 것 같습니다.

- C#으로 코딩테스트를 연습하면서 단순히 이직을 위함이 아니라 C#을 공부한 것을 접목시켜보고 문제 해결 능력 을 키워보려고 했습니다.
 - c++로 코딩테스트를 준비했을 때는 남는 게 없었습니다. 언어라는 게 큰 개념은 같지만 어쨌든 깊게 생각 할 때는 달랐고, 지식이 파편화되는 건 분명한 단점입니다.
 - C#으로 준비하다 보니, 난이도가 있는 문제에는 항상 저의 부족한 기초가 많이 보이게 되었습니다.
 - .Net Container와 LINQ가 편하니까 사용했지만 Array를 사용하지 않으니 시간초과가 났고,
 - StringBuilder를 쓰지 않고 String을 쓰면 매 번 새로운 문자열을 할당하다 보니 메모리 초과가 났 습니다.
 - struct 와 class가 스택이냐 힙이냐가 중요한 게 아니라, 결국에는 뭐가 메모리 덜 쓰고 뭐가 deepCopy니까 원본변경에 주의하고, 어떨 때 struct를 쓸 지 tuple을 쓸 지 class를 쓸 지를 고민 하게 됩니다. 그래야 정답이 되더라고요.
 - 기업 코테를 정말 많이 봤고 통과도 운이 좋게 많이 해봤었습니다. 근데 항상 이런 걸 왜 할까? 현업이랑 연결도 안 되는데 라는 생각을 많이 했습니다.
 - 하지만 현업에서 구현을 잘 하기 위해, 더 좋은 자료를 만들기 위해, 대용량 데이터를 빠르게 처리하기 위 해 이런 관점으로 문제를 풀다 보니 참 많은 걸 배웠습니다.
 - GitHub 링크
 - [GitHub_C# 코딩테스트 소스코드 모음](#)
 - [GitHub_C#](#)
 - [GitHub_알고리즘 전략](#)

- 시니어분들의 블로그를 읽고 정리하기 시작했습니다.
 - [🔗GitHub_Development Insight Journal](#)

3 달라진 점

- 26년도에 오프라인 필기테스트를 보고 왔습니다. 결과는 좋지 않았지만 깨달은 게 있습니다.
-

GitHub 링크

- [🔗GitHub_C#](#)
- [🔗GitHub_C# 코딩테스트 소스코드 모음](#)



네 번째 해결과정 : 메모 -> 기록

1 문제 재정의 + 어떻게 해결해야 할지

- 우리 코드 UI 어떻게 구성되었니? 어 나는 UI 쪽 코드 계속 봤고 메모도 했는데? 조금만 깊게 물어보시면 제대로 아는 게 하나도 없더라.
 1. 많이 적혀있지만 그게 어디 있는 지 나도 모른다.
 - 과거 문서 캡처 및 링크
- - 메모는 많이 한다. 그리고 어디에도 잘 저장한다. 그러나 돌이켜보면 다시 본 기억이 없다. 그러니 실력이 늘지 않았다. 링크나 캡처는 또 엄청 한다. 가공하기 귀찮으니까
- 1. 퇴사 직전 시점에 입사 이전보다 개발자로서의 경험이 늘었는가? 실력이 늘었는가? 실력에 대해서 늘었다고 생각을 못했고 무너졌던 거 같다.

2 해결 과정

1. 현실적인 직장인의 한계에 맞게 만들어 낸 기록 시스템
 - 회사에서 업무를 하면서 동시에 공부를 하기란 어렵다. 즉, 메모 -> 기록이 현실적으로 어렵다.
 - 하지만 패턴화 하면 된다.

오늘의 메모

• 개발 메모

◦ 질문

- ✓ 매니저가 controller에게 조회하기 위한 값을 controller가 필드로 캐싱해도 되는가?

◦ 메모

- GPT 대화 및 공부한 내용의 1~2줄 최종 요약만 적는다.

- 근거가 확실한 핵심 내용 & 키워드

- 링크

- ✓ InputController는 MonoBehaviour라서 아무 데서나 new 하기 어렵다.

InputController는 Manager에 등록된다.

외부 흐름은 Manager를 통해 Controller를 조작한다.

따라서 Controller의 public 메서드가 있어도 호출 경로를 제한하기 쉽다.

- ✓ Controller는 Unity 세계와 가까운 계산을 합니다.

- ✓ 근니까 controller와 manager가 모두 mousepointerpos를 들고 있는 건 괜찮다. 근데 반드사 manager를 통해 조회가 되고 manager는 그 조회를 위해 controller의 값을 가져온다. 그러면 콜백도 필요없다

- ✓ 컴파일 타임에 관계가 고정되어 있다

→ 개발자가 직접 적는다

런타임에 값/상태/데이터에 따라 대상이 바뀐다

→ 매핑, Dictionary, Factory, Registry 등을 고려한다

◦

- 개발 메모

- 질문



- `foreach (var pair in _dict.OrderBy(kv => kv.Key)) -> 캐싱 해?`

- 메모

- GPT 대화 및 공부한 내용의 1~2줄 최종 요약만 적는다|

- 근거가 확실한 핵심 내용 & 키워드

- 링크



- compile 타입에 개발자가 타입을 알면 Generic



- 필드 + 메서드에 공용으로 쓰는 generic은 class에 선언 메서드에서만 사용되는 generic은 method에만 선언!



- 좋은 추상화는 누구나 직관적으로 알 수 있어야 한다.



- 지금 딱 알았다. 확실한 거로 정리 Controller는 일단 자신을 GameManager에게 전달하니 문제 X 문제는 연결할 Manager 객체임 싱글턴. 그니까 new도 안 먹히고, 근데 나는 컴파일 타입에 그 Manager의 타입을 안다. 어어어어 타입???? 리플렉션??? 아 있는데 생각해보니까 GameManager에서 Manager들의 타입을 알면 인스턴스를 리턴하는 메서드 만들면 되잖아. 그러면 두 인스턴스 알고 호출도 되고! 나 이제 이해한 듯

-

- 우선 이렇게 노선에 적는다. gpt를 통해 공부하거나 코드를 읽으며 모르는 개념을 구글링하다보면 핵심 키워드가 생기고 그걸 적는다.

-

2. 구현과의 병행

3 달라진 점

 GitHub 링크

- 



마지막 해결과정 : 다시 일하기 위한 나만의 지도

1 문제 재정의 + 어떻게 해결해야 할지

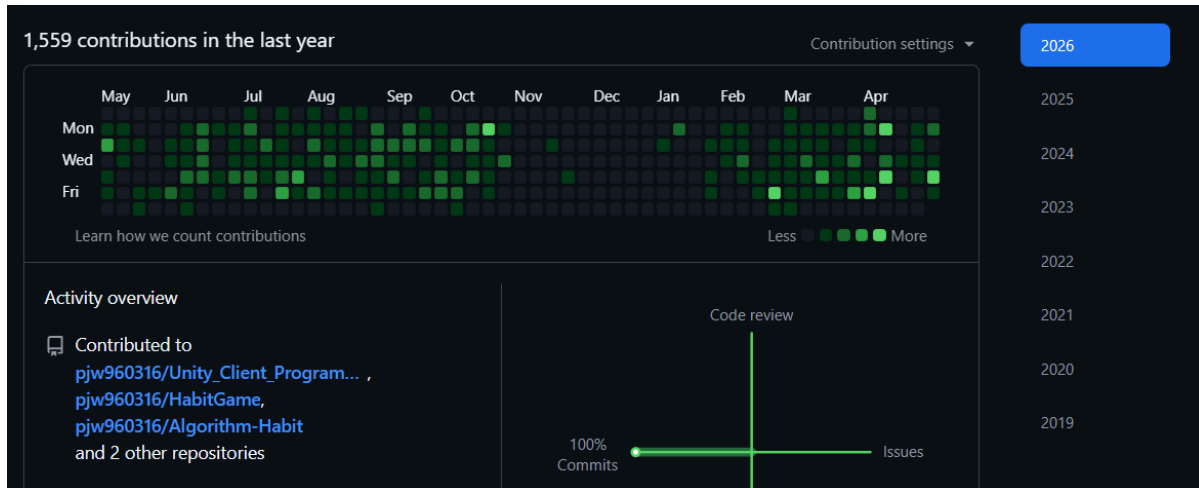
- 2024년, 거의 집에만 있으며 몸과 마음이 무너졌었습니다. 병원도 가볼까 했었습니다. 전화도 했었고요. 근데 가는 게 무서워서 안 갔습니다. 지금도 항상 운이 좋았다고 생각합니다. 그리고 친구들과 부모님 그리고 유튜브 GPT 덕분에 어찌어찌 다시 나가보게 되었습니다. 불암산을 끌려 간적이 있습니다. 당시에 거의 100kg가 되었지요. 30번 쉬었지만 올라갔어요. 그 때 좀 많은 걸 느꼈습니다. 그리고 친구들과 일본 여행도 다녀왔습니다. 그렇게 다시 무언가를 해보려고 했어요.
- 2025년, 다시 하나씩 천천히 해봤습니다. 24년에 시간이 많아서 책은 참 많이 읽었어요. 그러면서 배운 건 그냥 쉬운 걸 하나씩 해보자. 몸이 정신보다 중요하다. 헬스장 등록만 하고 안 가본 기억이 많았죠. 매 번 운동 프로그램만 계획하고. 지쳐서 그냥 안 가고. 근데 그냥 한 두달은 러닝머신만 탔습니다. 그러다보니 많이 나아지더라고요. 매일 오전에 일어나서 샤워부터 시작했습니다. 그것도 어려웠고요. 그러나 점차 나아지니 샤워도 하고, 처음에는 동네만 다니다가 어느 순간 도서관도 가보고, 내가 왜 다시 개발하는 게 무서웠는지도 계속 시도하며 나아졌습니다. 그러다보니 주 2번 나가던게 3번이 되고 4번이 되고 결국 주중에 매일 나가게 되었습니다. 뭘 계속 해보다 보니 좀 더 좋은 방향이 보이고 제가 그런 걸 하나씩 달성하다보니 스스로 즐거운 게 느껴지더라고요.
- 2026년, 이직도 도전해보았습니다. 많이 탈락도 해봤고, ai시대라 급변도 했고요. 탈락하면 여전히 마음은 아픕니다. 그러나 이제는 그냥 걸거나, 카페가서 생각을 정리하거나 하는 저만의 방패도 생겼어요. 꼭 잘 먹고 일찍 자고요. 지금은 주중에는 매일 정해진 시간에 나갑니다. 그러지 않으면 집에 눕는 다는 걸 2025년에 많이 알았거든요. 뭐라도 하루에 하나는 배웁니다. 꾸준히 뭐라도 하다보니 문제 해결력이 생깁니다. 루틴이 잡히고요.

2 해결 과정

3 달라진 점

후기

- 이제는 슈팅게임을 혼자 만들 수 있을 것 같아요. 아니 어떤 게임도 혼자서 만들 수 있을 것 같습니다.
- 하지만 부족한 사람이라 혼자서 하기에는 벅차고 배움이 적어요
- 그래서 조금은 달라진 사람으로 회사에서 팀에 기여해보고 싶습니다.



2025년 11월 ~ 2026년 1월은 이직 서류 준비, 필기 시험 준비를 병행하던 시기입니다. 그러나 이 때도 게임 개발을 꾸준히 했어야 했습니다.

긴 시간 읽어주셔서 감사합니다.

글감 TMP

문제 해결을 위해 기록한 내용

- 필기테스트 보고 온 후기 -> 아 뭔가 더 같고 닳으면 100점 맞을 수 있겠는데? 아쉽지만 30점 맞던거 90점 된거 같다.
- 지금이야 내가 온전히 시간을 관리하니 가능할지 모른다. 과연 회사를 다니면서도 이 프로세스가 정상으로 동작 할까? 과거처럼 무너지지 않을까? 하지만 이젠 확신할 수 있다. 적어도 그 상황에 맞는 최선의 해결책을 찾는 능력은 생긴 것 같다. 또 수정하고 또 고치면 된다. 하다보면 바뀐다.

포폴도 업데이트 관리 주체

- 매 번 pdf로 만들고 방치 -> 6개월 뒤 업데이트 -> 방치
- 그러나 이제는 markdown으로 관리도 쉬우니 계속 업데이트할 수 있다.